

EKS + Node Group + Add-ons + IRSA

- module "eks" [T]
- Terraform module ← ال Module في Terraform
 - شبه ال Function في البرمجة ← بدل ما تكتب 300 Resources بنفسك، حد كتبهم قبل كذا وجمدهم في Module انت بس بتناديه بس
 - بيبن لي Kubernetes Cluster كامل على AWS
 - بدل ما أعلن ← IAM Role * SG * Node Groups * Addons * EKS Cluster
 - كل دا بيدي وي بيحل مشكلة أليه ؟
 - لو عملت EKS بيدي وي هحتاج أكتب أكثر من 60 Terraform Resources لكن بال Module خلاص
 - بمضيف ال Repo الرسمي لل module "terraform" Source = "terraform"
 - و Terraform هيروح يحمل الكود منه
 - كأني عامل Git Clone لكن Terraform بيحمله الوصلة
 - لأن بيحدد Version 20.x.x.x لأن بيغير APIs ممكن
 - Version 21 بيحل Variable فتلاقي المشروع كله باق
 - بمضيف اسر لل Cluster
 - بتفصيلة ال VPC
 - بتحدد ال Worker Node داخل ال Private Subnet
 - ال IRSA ← enable-irsa=true دامن اهو السطور
 - IAM Roles for Service Accounts
 - زمان كان كل Pod بيافد صلاحية ال EC2
 - يعني لو Node عنده Amazon S3 Full Access هيقدر يمشي كل ال Pods على الجهار يقدر يدخل على ال S3
 - الحل كان في ال IRSA ← كل Pod بيحصل
 - ليه ال IAM Role الخاصة بيه ← Valero
 - * EBS CSI * ELB Controller

module "eks" {

EndPoint ← Api Server

Cluster_endPoint_Public_access = false

ببطل ال Public EndPoint الخاص بال Kubernetes API

يعني ال Api Server مش هيقدر يوصل ال Pub IP على الإنترنت

ال API موجود جوا ال VPC فقط

Cluster_endPoint_Private_access = true

يعمل ال Private EndPoint داخل ال VPC يعني أي Resource

داخل نفس ال VPC يقدر يكلم ال API

enable_Cluster_Creator_admin_Permission = true

نشان لما كنت تعمل ال Cluster

ال IAM User اللى عمل ال Cluster

مش ضروري يعني Admin وكان لازم اعمل

aws-auth ConfigMap واضيف نفسي

دلو في AWS اضافت خاصية اول شخص

يعمل ال Cluster ياخد Cluster Admin تلقائي

ال Logs

Cluster_enabled_log_types = [api, audit, auth]

دا بيحدد انواع ال Control Plan Logs اللى AWS

تخبرها ال Cloud Watch

هو موجود عنوان ال Control Plan في EKS

منار بواسطة AWS مشتركة دخل السيرفرات بتاعة

بالتالي الطريقة اللى اعرف بيها في ال Cluster

هي ال Logs

module "eks" [1]

KMS

Encryption

في Kubernetes في Resources كثيرة جداً
ال etcd عبارة عن DB كل مكونات
ال Cluster موجودة فيها
لو عملت Secret طبقة من امان

أي حد يقرر يعمل

وهيطلع ال Pass -d base-64
بالتالي ال Secret بتخزن بدون Encryption

هنا يجي دور ال KMS

AWS Service اسمها Key Management Service
وظيفةها ادارة المفاتيح التشفير

بدل ما انت تنشئ مفاتيح AWS بيطلع داتاها عن

Create_Kms_Key = true Terraform هيطلع ال Kms Key
مخزونة ال Cluster

ليه بقى لأن Kubernetes محتاج مفتاح يشفر فيه
البيانات من غير مفتاح عن Encryption

```
Cluster-encryption-Config = {  
  resources = ["secrets"]
```

بيقول ال KMS لما يجي تخزن Secret

داخل ال etcd اسخدم مفتاح ال KMS وشفرها

يعني طبقة Resources تقرر نوع

Resources

ال API Server هو اللي يطلبه فك التشفير من KMS

ويعرف النتيجة للمستهلك الصحيح له

بيحل مشكلة بدل ما البيانات الحساسة

تتخزن جوا ال etcd ممكن قرائتها بتخزن مشفرة
جوا ال etcd

هو بيحفر في Kubernetes Secrets

module 'eks'

eks-managed node-groups ← هذا المكان الذي يتعرف فيه ال Worker Node و AWS في الذي يديره عنك ال AutoScaling, Upgrades
بمضيف نوع ال EC2 + كم عدد ال Worker و ال Max

Cluster Addons ← يعرفها ويبدأ اكتب

كل ال Addons التي تحتاجها فقط على مستوى ال AWS Managed Addons

اما باقي ال Addons يكتبها في ملف خارجي

Coredns ← عندي مثلاً DB ← IP = 10.0.2.15

ال DB اتصلها Restart وبقى IP = 10.0.7.31
ال حل: كان في ال Coredns ال APP يكتب service db

Kube-Proxy ← عندي مثلاً Service و 3 Pods

ازالة ال APP في ال Pod من ال Cluster
دورة ال يعمل Load Balancer جوا ال Cluster و بيتم ال Kubernetes services ال

VPC-CNI ← يعني Container Network Interface

ال Pod لما اتصل ال CNI هو الذي يدي ال Pod ال IP بكامها ← هو بيوز ال EC2 ياخذ IP من ال ENI و يديها ال Pod

module "eks"

Addons في عتبات نوعين من ال

AWS Managed Addons

دي Addons مدمومة أو مكتوبة من AWS

و AWS هي اللي بتدير ال Lifecycle بناعها

* Install

* Updates

يعني AWS هي المسؤولة عن

لما نكتب ال Cluster-addons

Coredns =

Kub-Proxy =

هنا Terraform بيكلم ال AWS API

و ال AWS هي اللي تضيف ال Addons

انته من بتعمل Helm install

Self-Managed Add-ons

دي AWS ملهاش علاقة بيها

انا اللي بنجيبها و عندي طريقتين

الطريقة اليدوية ادخل واصل ال Addons من جوا

ال EC2 اضيف ال VPC و اعمل Install

بس هنا Terraform من هي تعرف أي حاجة

اللي بنسخدمها في ال Production

بنكتب كود Terraform و ال Terraform

يرج يقول لـ Helm حمل ال addon

شروط اساس ال Helm يكون مكتبة عندك

عند ال Terraform يقدر ينزل ال Addons

هناك ملف خاص بال Self-Managed Addons



IRSA = IAM Roles for Service Accounts [9]

نمان لو Pod عاين يتعامل مع AWS
مثلا يعمل ALB, EBS Volume, S3
الحل الاول ودا كان غلط

نبي لا EC2 صلاحية كبيرة
يعني كل Pod على ال Node يقدر يستعمل
نفس ال IAM Role يعني ال Front و ال Back
كلهم بياضوا نفس ال IAM Role
و دا مخالف لمبدأ ال Least Privilege

الحل الثاني وهو ال IRSA
بدلنا ال IAM Role يكون مربوط بال EC2
يعني مربوط بال Service Account
يعني كل Pod ياخذ Role خاصة بيه

module "lb-role" { ← module

دا Terraform Module بيعمل IAM Role
لكن مش في Role ده Role مخصوص ال IRSA

Source = "terraform-aws-

بقولة استخدت Module جاهزة من Terraform Registry

اسمها iam-role-for-service-accounts-eks ←

بنحدد ال Version ←

بنحدد اسم ال Role ←

attach L-b-c-Policy = true يعني كل ال Permission

الناشطة ال LB Controller بدلنا ان اطيحها
ال Permission

IRSA [2]

module "role" {

OIDC_Providers = { ← في سطر مهم جداً
وهو جزء في ال IRSA كله

OpenID Connect = OIDC ←

في ال EKS, ال OIDC هو اللي يسمح لـ AWS

يتم في هوية ال Service Account داخل ال Cluster

Kubernetes يقول ال Pod ده شغال باستخدام Service Account

اسم ال Controller Load-Balancer

AWS يراجع ال Provider OIDC

لو الهوية درجة يسمح لـ Pod انه يعمل Assume

لـ IAM Role ويدون OIDC هوية طريقة

AWS تتأكد بيها ان اللان ده جاي من Service Account

مسموح داخل ال Cluster

في سطر مهم Provider arn = module.eks.oidc.Provider.arn

احنا كنا علنا enable_irs = true

جوا Module ال EKS داخل Terraform يتنسخ OIDC

هنا نستفيد ال ARN بكرة

في هو ال ARN

Amazon Resource Name هو اختصار لـ

وهو عبارة عن معرف فريد لكل Resource في AWS

بصي كل Resource في AWS ليه ARN خاص بيه

علشان AWS تعرفه بشكل دقيق

namespace_service_accounts = kube-system

: AWS-loadBalancer Controller

بنحدد ال Namespace ونحدد ال Service Account



3) module "ebs_csi_driver"

Module بندي صلاحية للـ EBS CSI Driver
الـ EBS هو عبارة عن Hard Disk في AWS
لكن الـ EBS CSI Driver مش الهارد نفسه ده مجرد Driver
بيشغلي Kubernetes يعرف يتعامل مع الـ EBS
لأن Kubernetes مش عارف كذا AWS هو مجرد Orchestrator
لما الـ Pod يحتاج 20GB ساعته Kubernetes
مش هيعرف يجيبها منين لأنه مش عارفش يعني أي EBS
ولا يعرف يعمل Create Volume و Attach Volume لازم يعمل دا
صناعي دور الـ EBS CSI Driver ← **Container Storage Interface**
و دا Standard مدمج عنك أي Storage يقدر يتعامل
مع Kubernetes ← يعني في Driver لكل Storage
EBS CSI Driver ← **AWS EBS**
Azure CSI Driver ← **Azure Disk**
EFS CSI Driver ← **AWS EFS**
كلهم بيكلموا Kubernetes بنفس الطريقة
يبقى الـ EBS CSI Driver بيحصل أي ؟
لما نعمل PVC
الـ Driver يروح بيكلم API ويرجلك
بالـ Create EBS
بعد ما الـ Volume يتعمل فالـ Driver
هو اللي يعمل Delete, Attach, Create
وكل دا Automatic
مثلاً ← على MySQL Pod Deploy
وانا كتبت 20GB Storage بشكل تلقائي
هو يعمل الـ Volume

3] module "eks-csi-role"

ليه هو محتاج IAM Role ← لان ال Driver بيستخدم AWS API ولو مكنوش صلاحية هيرفض من AWS على ان كذا كنبنا .

attach-abs-csi-policy=true
بعولة اربط ال IAM Policy الرسمية الخاصة بـ Amazon EBS CSI Driver

ال Policy دي فيها صلاحية زي

*ec2: CreateVolume *ec2: DeleteVolume *

4] resource "aws-eks-addons" ebs-csi

ليه مكننا هون جوا ClusterAddons
لانا محتاجين ServiceAccountRole وده من فيكون

جاهز عالا بعد انشاء ال IAM Role
ولو مكننا جوا ال EKS هيعمل Dependency Cycle
وال IAM Role دي هي بتاعة ال csidriver
لو مكننا جوا ال ClusterAddons فيبقى عايز ال IAM Role
وال IAM Role من فيبقى جاهزة غير لما ال EKS بيبدأ

ليه محتاج IAM Role عشان يقدر بعد كذا يوزع
يعمل EBS لانه بيستخدم ال API

والله ال Terraform هيرفض يستعمل لانه فيكون
ال Dependency Graph فله وده من فيكون
الله لمرحلة ال IAM Role

طلب ليه على ServiceAccount عشان ال Pods
تكام ال API بتاعة AWS عشان تمشي PVC
و AWS ميعرفش ال Pods

EBS-CSI-Driver

[4]

depends_on = [module.eks]

بعض كان Module ال EKS يخلص قبل
ما Terraform يبدأ في إنشاء ال addons

EKS Cluster

↓
OIDC Provider

↓
IRSA Role
(ALB)

↓
ALB Controller

↓
AWS API

↓
IRSA Role
(EBS CSI)

↓
EBS CSI Driver

↓
AWS EBS API

Done
EKS